



Machine Learning the Voltage of Electrode Materials in Metal-ion Batteries

RESEARCH PAPER IMPLEMENTATION

Materials Science + Machine Learning + Regression

DNN · SVR · KRR · PCA Feature Reduction · Voltage Profile Prediction

Dear Students,

Follow the steps below to implement the research paper. Your objective is to develop a machine learning system that predicts the voltage of electrode materials in metal-ion batteries using material features and machine learning regression algorithms. This guide converts the paper's methodology into a practical implementation workflow for your Crystal Domain project.

Prepared for

Sreenivas

Assistant Professor, CSE (AI & ML) · Mentor, Crystal Domain

1. Understand the Research Paper

Objective

Predict the average voltage of electrode materials used in metal-ion batteries using machine learning.

The original paper uses data from the Materials Project Database, material and elemental properties, and three machine learning approaches:

1. Deep Neural Network (DNN)
2. Support Vector Regression (SVR)
3. Kernel Ridge Regression (KRR)

The predicted voltage is compared with reference values to evaluate model performance.

Input → Processing → Output

Electrode Material Info (Composition, Ion, Lattice, Group)	Feature Extraction 237 numerical features	Normalization + PCA 237 → 80 PCs	ML Regression DNN · SVR · KRR	Predicted Average Voltage (V)
---	--	-------------------------------------	----------------------------------	----------------------------------

⚠ IMPORTANT

The paper reports 237 features and retains 80 principal components for model training. Your team must reproduce the feature definitions accurately rather than inventing a different set of 237 features.

2. Complete Project Workflow

Follow these steps in the given order.

1	Understand the paper Study the objective, data, features and algorithms.
2	Collect the dataset Obtain electrode material voltage records.
3	Clean and inspect data Remove inconsistencies, repetitions and invalid records.
4	Generate features Convert material information into numerical features.
5	Normalize + PCA Scale features and reduce dimensions.
6	Train regression models DNN, SVR and KRR.
7	Evaluate the models Use MAE and additional regression metrics.
8	Build the prediction system Predict voltage for new electrode materials.
9	Implement an improvement Introduce and evaluate a clearly documented enhancement.

3. Step 1 – Set Up the Development Environment

Tools Required

- Python 3.10 or 3.11
- Jupyter Notebook or Google Colab
- VS Code (optional)
- GitHub for project code
- Materials Project API (if using API-based data collection)
- TensorFlow/Keras for DNN
- Scikit-learn for SVR, KRR, PCA and evaluation

Install Required Libraries

Open Google Colab or your terminal and run:

```
pip install numpy pandas matplotlib seaborn  
pip install scikit-learn tensorflow  
pip install pymatgen mp-api joblib
```

Create a requirements.txt file:

```
numpy  
pandas  
matplotlib  
seaborn  
scikit-learn  
tensorflow  
pymatgen  
mp-api  
joblib
```



DELIVERABLE

Screenshot of the working environment and installed libraries.

4. Step 2 – Download or Obtain the Dataset

Dataset Information from the Paper

Item	Paper information
Source	Materials Project Database
Initial data instances	4,250
Electrode materials	3,580
Retained after cleaning	3,977
Target	Average electrode voltage
Battery ions	Li, Mg, Ca, Al, Zn and Y
Feature count	237
Reduced feature count	80 principal components

What Students Must Do

Option A – Reproduce the paper's dataset

Obtain the original dataset or a reliable supplementary dataset containing the voltage records and required material information.

Option B – Implement an adapted version

Use current Materials Project data or another documented dataset, clearly recording the differences from the original paper.

i NOTE

The paper's reported 3,977 retained records should not be assumed to be directly reproducible through a current API query. Confirm the dataset version and extraction procedure before claiming exact reproduction.

Expected Dataset Columns

Your dataset should contain, where available:

Column	Description
Material ID	Unique material identifier
Composition	Chemical formula
Working ion	Li, Na, K, Mg, etc.
Low-concentration composition	Initial electrode composition
High-concentration composition	Intercalated composition

Column	Description
Ion concentration	Relevant concentration information
Crystal lattice	Crystal lattice type
Space group	Space group number
Voltage	Average voltage in volts

The paper uses the working ion, active ion concentration, crystal lattice type and space group, along with elemental properties, as part of the feature representation.

5. Step 3 – Inspect the Dataset

Create a Jupyter Notebook:

```
01_dataset_inspection.ipynb
```

Load the Dataset

```
import pandas as pd

df = pd.read_csv("battery_voltage_dataset.csv")

print(df.head())
print(df.shape)
print(df.columns)
print(df.info())
```

Check Missing Values

```
print(df.isnull().sum())
```

Check Duplicate Records

```
print("Duplicate rows:", df.duplicated().sum())
```

Check Voltage Distribution

```
import matplotlib.pyplot as plt

plt.hist(df["voltage"].dropna(), bins=30)
plt.xlabel("Voltage (V)")
plt.ylabel("Frequency")
plt.title("Voltage Distribution")
plt.show()
```

Student Observation

Record the following:

- Total number of rows
- Total number of columns
- Missing values
- Duplicate records
- Minimum and maximum voltage
- Distribution of working ions

 DELIVERABLE

Dataset inspection report with screenshots and observations.

6. Step 4 – Clean the Dataset

The original paper removed inconsistencies and repetitions, retaining 3,977 data instances from the initial 4,250.

Cleaning Tasks

1. Remove duplicate records.
2. Handle missing feature values.
3. Check invalid voltage values.
4. Verify composition and working ion information.
5. Check whether the same material has multiple concentration intervals.
6. Ensure the target voltage corresponds to the correct electrode record.

Example Code

```
df = df.drop_duplicates()

df = df.dropna(subset=["voltage"])

df = df[df["voltage"].notna()]

print(df.shape)
```

⚠ IMPORTANT

Do not blindly remove multiple voltage records for the same material. The paper explains that one electrode can have multiple average voltage instances for different intercalation concentration ranges.

📦 DELIVERABLE

Cleaned dataset saved as data/cleaned_voltage_dataset.csv

7. Step 5 – Generate Numerical Features

This is one of the most important steps in your research implementation.

The machine learning model cannot directly process chemical formulas and material descriptions as raw text. You must convert the material information into numerical features.

Feature Categories

Category	Description
A. Elemental Properties	Properties of the elemental constituents of the electrode material, based on the paper's feature approach.
B. Structural Features	Crystal lattice type and space group number.
C. Battery & Composition Features	Working ion and concentration of the active metal ion.

The original paper states that elemental properties are adopted from the work of Ward et al. and combined with composition and structural information.

Implementation Requirement

Your team must prepare a feature-generation module.

Suggested file: src/feature_engineering.py

```
def extract_elemental_features(composition):
    pass

def extract_structural_features(material):
    pass

def build_feature_vector(material):
    pass
```

⚠ IMPORTANT

The above is a coding structure, not the complete feature extraction algorithm from the paper. You must implement and document the exact feature definitions you choose.

Expected Output

Raw material information	Feature extraction	Numerical feature matrix	237 features (if reproducing paper)
--------------------------	--------------------	--------------------------	-------------------------------------

Before proceeding, verify:

```
print(X.shape)
```

For a faithful reproduction of the paper's stated representation, the feature matrix should have 237 columns before PCA. If your dataset or feature definitions differ, report the actual feature count.

8. Step 6 – Normalize the Features

The paper normalizes input features to approximately the range -1 to $+1$ during training and does not normalize the target voltage.

For a practical scikit-learn pipeline, you can use a scaler fitted only on the training data.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)
```

Important Rule

⚠ IMPORTANT

Do not fit the scaler on the complete dataset before splitting into training and testing. That would allow information from the test set to influence preprocessing.

If reproducing the original paper exactly, document the difference between its on-the-fly scaling approach and your chosen implementation.

DELIVERABLE

data/normalized_features.csv

9. Step 7 – Apply PCA

The paper applies Principal Component Analysis (PCA) to remove redundancy and reduce the feature dimensionality. It reports that 80 principal components were used for model training.

PCA Workflow



Example Code

```
from sklearn.decomposition import PCA

pca = PCA(n_components=80)

X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

print("Training shape:", X_train_pca.shape)
print("Testing shape:", X_test_pca.shape)
```

Plot Explained Variance

```
import numpy as np
import matplotlib.pyplot as plt

cumulative_variance = np.cumsum(pca.explained_variance_ratio_)

plt.plot(range(1, len(cumulative_variance) + 1),
         cumulative_variance)

plt.xlabel("Number of Principal Components")
plt.ylabel("Cumulative Explained Variance")
plt.title("PCA Explained Variance")
plt.grid(True)
plt.show()
```

i NOTE

Verify that 80 components provide an appropriate representation for your dataset. The paper's 80-component choice is a reported methodology, not a guarantee that the same choice is optimal for a modified dataset.

10. Step 8 – Split the Dataset

The paper uses:

- 90% Training Set (T-set)
- 10% Holdout Test Set (H-set)
- 10-fold cross-validation on the training set

The holdout set is not used for model training.

Example Code

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.10,
    random_state=42
)
```

⚠ IMPORTANT

Perform the train/test split before fitting the scaler and PCA. Fit those preprocessing steps using the training data only.

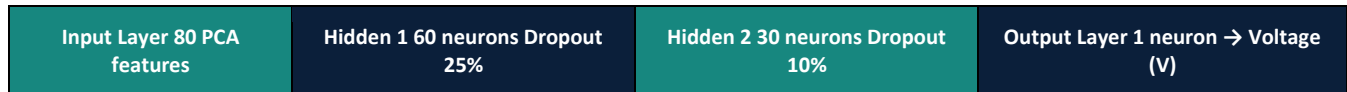
For a more robust implementation, use a pipeline to prevent data leakage during cross-validation.

11. Step 9 – Implement DNN Regression

The paper uses a four-layer DNN architecture with the following reported structure:

Layer	Nodes / Configuration
Input layer	80
First hidden layer	60
Second hidden layer	30
Output layer	1
Dropout	25% and 10%
Regularization	L2
Loss	Mean Squared Error
Metric	Mean Absolute Error
Optimizer	RMSprop

Architecture Diagram



Example TensorFlow/Keras Implementation

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.regularizers import l2

model = Sequential([
    Dense(60, activation="relu",
          kernel_regularizer=l2(0.001),
          input_shape=(80,)),

    Dropout(0.25),

    Dense(30, activation="relu",
          kernel_regularizer=l2(0.001)),

    Dropout(0.10),

    Dense(1)
])

model.compile(
```

```
optimizer=tf.keras.optimizers.RMSprop(),  
loss="mse",  
metrics=["mae"]  
)  
  
model.summary()
```

i NOTE

The code above is an implementation starting point. Confirm the activation functions and exact regularization strength used in the original paper's supplementary information before claiming an exact reproduction.

Train the Model

```
history = model.fit(  
    X_train_pca,  
    y_train,  
    validation_split=0.1,  
    epochs=100,  
    batch_size=32,  
    verbose=1  
)
```

Predict Voltage

```
y_pred_dnn = model.predict(X_test_pca).ravel()
```

12. Step 10 – Implement SVR Regression

Support Vector Regression is used as a comparison model in the paper.

The paper uses:

- RBF kernel
- Grid search for C and gamma
- 10-fold cross-validation

The reported optimal values for the complete dataset were $C = 10.0$ and $\gamma = 0.1$.

Example Code

```
from sklearn.svm import SVR

svr_model = SVR(
    kernel="rbf",
    C=10.0,
    gamma=0.1
)

svr_model.fit(X_train_pca, y_train)

y_pred_svr = svr_model.predict(X_test_pca)
```

Optional Grid Search

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    "C": [0.01, 0.1, 1, 10, 100],
    "gamma": [0.001, 0.01, 0.1, 1]
}

grid_svr = GridSearchCV(
    SVR(kernel="rbf"),
    param_grid,
    cv=10,
    scoring="neg_mean_absolute_error",
    n_jobs=-1
)

grid_svr.fit(X_train_pca, y_train)

print(grid_svr.best_params_)
```

13. Step 11 – Implement KRR Regression

The paper uses Kernel Ridge Regression with an RBF kernel.

The reported optimal parameters for the complete dataset were:

- $\alpha = 0.01$
- $\gamma = 0.1$

Example Code

```
from sklearn.kernel_ridge import KernelRidge

krr_model = KernelRidge(
    kernel="rbf",
    alpha=0.01,
    gamma=0.1
)

krr_model.fit(X_train_pca, y_train)

y_pred_krr = krr_model.predict(X_test_pca)
```

i NOTE

Student task: compare DNN, SVR and KRR using the same test set and evaluation metrics.

14. Step 12 – Evaluate Model Performance

The primary metric used in the paper is Mean Absolute Error (MAE).

Formula

$$\text{MAE} = (1/N) \sum |Y_i \text{ target} - Y_i \text{ predicted}|$$

Python Code

```
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)
import numpy as np

def evaluate_model(y_true, y_pred, model_name):
    mae = mean_absolute_error(y_true, y_pred)
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_true, y_pred)

    print(f"\n{model_name}")
    print(f"MAE: {mae:.4f} V")
    print(f"RMSE: {rmse:.4f} V")
    print(f"R2 Score: {r2:.4f}")

    return {"Model": model_name, "MAE": mae, "RMSE": rmse, "R2": r2}
```

Evaluate All Three Models

```
results = []

results.append(evaluate_model(y_test, y_pred_dnn, "DNN"))
results.append(evaluate_model(y_test, y_pred_svr, "SVR"))
results.append(evaluate_model(y_test, y_pred_krr, "KRR"))
```

Compare Results

```
results_df = pd.DataFrame(results)

print(results_df)
```

Paper Reference Results

The paper reports the following performance values for its complete dataset:

Model	Mean MAE (10-fold T-set)	H-set MAE
DNN	0.43 ± 0.03 V	0.43 V
SVR	0.42 ± 0.13 V	0.40 V
KRR	0.46 ± 0.14 V	0.39 V

⚠ IMPORTANT

These are the paper's reported results, not the expected scores for your implementation.

15. Step 13 – Plot Actual vs Predicted Voltage

Create scatter plots to analyze the relationship between the actual and predicted values.

Example Code

```
import matplotlib.pyplot as plt

def plot_predictions(y_true, y_pred, model_name):
    plt.figure(figsize=(7, 5))
    plt.scatter(y_true, y_pred, alpha=0.6)

    min_value = min(y_true.min(), y_pred.min())
    max_value = max(y_true.max(), y_pred.max())

    plt.plot([min_value, max_value], [min_value, max_value], linestyle="--")
    plt.xlabel("Actual Voltage (V)")
    plt.ylabel("Predicted Voltage (V)")
    plt.title(f"{model_name}: Actual vs Predicted")
    plt.grid(True)
    plt.show()

plot_predictions(y_test, y_pred_dnn, "DNN")
plot_predictions(y_test, y_pred_svr, "SVR")
plot_predictions(y_test, y_pred_krr, "KRR")
```

Required Figures

- Actual vs predicted voltage – DNN
- Actual vs predicted voltage – SVR
- Actual vs predicted voltage – KRR
- Model comparison graph
- Training and validation loss curves for DNN

The original paper includes scatter plots comparing target and predicted voltage values for the holdout set (Figure 4, page 10 of the uploaded PDF).

16. Step 14 – Implement Voltage Profile Prediction

★ ADVANCED

This is an advanced implementation component.

The paper uses ML models to generate a voltage profile diagram by predicting voltages at different intercalation concentrations. The example in Figure 6 compares DNN, SVR, KRR and DFT values for the sodium-based electrode $\text{Na}_x\text{Co}_2\text{SbO}_6$.

Workflow

Select Electrode Material	Generate Different Ion Concentrations	Extract Features per Composition	Predict Voltage per Concentration	Plot Concentration vs Voltage
---------------------------	---------------------------------------	----------------------------------	-----------------------------------	-------------------------------

Example Output

Ion concentration (x)	Predicted voltage (V)
0.0	Model prediction
0.5	Model prediction
1.0	Model prediction
1.5	Model prediction
2.0	Model prediction

⚠ IMPORTANT

The above concentrations are an example of how to structure the experiment. Use valid compositions and concentration ranges for your selected electrode material.

17. Step 15 – Predict Voltage for New Materials

After completing the training and evaluation, develop a prediction function.

Input

- Electrode composition
- Working ion
- Ion concentration
- Crystal lattice type
- Space group number
- Required elemental features

Output

- Predicted average voltage in volts

System Workflow



Example Function Structure

```
def predict_voltage(material_features, scaler, pca, model):
    scaled_features = scaler.transform(material_features)
    pca_features = pca.transform(scaled_features)
    predicted_voltage = model.predict(pca_features)
    return predicted_voltage
```

⚠ IMPORTANT

The prediction input must use the same feature definitions, column order, scaling and PCA transformation that were used during model training.

18. Step 16 – Research Improvement

Your project should not stop at reproducing the basic machine learning models. Each team should select a clearly defined improvement.

Possible Improvements

Improvement	Description
1. Feature Engineering	Investigate an alternative, documented set of composition or structural descriptors and compare performance.
2. Hyperparameter Optimization	Compare systematic tuning strategies for DNN, SVR or KRR.
3. Additional Regression Models	Evaluate a new regression algorithm and compare it with the paper's models.
4. Battery-Ion Comparison	Study model behavior across different working ions, with careful validation and appropriate data.
5. Model Explainability	Analyze the contribution of features or transformed components, while recognizing that PCA components are not directly equivalent to original material properties.

The paper itself identifies additional data, alternative algorithms and other ways of representing intercalation reactions as possible directions for improvement.

19. Suggested Project Folder Structure

```
ML_Battery_Voltage/
├── data/
│   ├── raw_voltage_dataset.csv
│   ├── cleaned_voltage_dataset.csv
│   └── processed_features.csv
├── notebooks/
│   ├── 01_dataset_inspection.ipynb
│   ├── 02_feature_engineering.ipynb
│   ├── 03_normalization_pca.ipynb
│   ├── 04_dnn_model.ipynb
│   ├── 05_svr_model.ipynb
│   ├── 06_krr_model.ipynb
│   └── 07_model_comparison.ipynb
├── src/
│   ├── feature_engineering.py
│   ├── preprocessing.py
│   ├── models.py
│   └── evaluation.py
├── models/
│   ├── dnn_model.keras
│   ├── svr_model.joblib
│   ├── krr_model.joblib
│   ├── scaler.joblib
│   └── pca.joblib
├── results/
│   ├── metrics.csv
│   ├── plots/
│   └── predictions.csv
├── requirements.txt
├── README.md
└── app.py
```

20. Eight-Day Student Implementation Plan

Use this schedule to organize your implementation work.

DAY 1 Paper Understanding & Environment

- Read the paper objective and methodology.
- Understand electrode materials, voltage and intercalation.
- Set up Python and required libraries.
- Create GitHub repository and folder structure.

DAY 2 Dataset Collection & Inspection

- Identify the dataset source and version.
- Obtain voltage and material information.
- Inspect columns, missing values and duplicates.
- Prepare a dataset report.

DAY 3 Data Cleaning & Features

- Clean inconsistent and duplicate records.
- Understand the 237-feature representation.
- Implement or document feature extraction.
- Generate numerical feature matrix.

DAY 4 Normalization & PCA

- Split training and holdout test data.
- Fit the scaler on training data.
- Apply PCA and inspect explained variance.
- Prepare model-ready data.

DAY 5 DNN Implementation

- Implement the DNN architecture.
- Train the regression model.
- Track loss and MAE.
- Save the model and preprocessing objects.

DAY 6 SVR & KRR Implementation

- Implement SVR with RBF kernel.
- Implement KRR with RBF kernel.
- Perform hyperparameter tuning.
- Compare regression predictions.

DAY 7 Evaluation & Voltage Profiles

- Calculate MAE, RMSE and R^2 .
- Generate actual vs predicted plots.
- Test on a separate holdout set.
- Implement a voltage profile experiment if the required data is available.

DAY 8 Improvement & Final Demonstration

- Implement your team's research improvement.
- Compare baseline and improved results.
- Prepare architecture diagram and project documentation.
- Record a demonstration and finalize the presentation.

21. Final Deliverables for Each Team

Every team must submit the following:

- ☐ Research paper understanding and summary
- ☐ Dataset source and collection documentation
- ☐ Cleaned dataset
- ☐ Feature extraction code
- ☐ Normalization and PCA implementation
- ☐ DNN model
- ☐ SVR model
- ☐ KRR model
- ☐ Model performance comparison
- ☐ Actual vs predicted graphs
- ☐ Research improvement
- ☐ GitHub repository
- ☐ Final architecture diagram
- ☐ Working demonstration
- ☐ Final presentation and implementation report

22. Final Architecture for Your Presentation

Architecture Components

1. Data Collection – Materials Project / documented dataset.
2. Data Preprocessing – Cleaning and validation.
3. Feature Engineering – Composition, elemental and structural properties.
4. Feature Scaling – Normalize input features.
5. PCA – Reduce 237 features to 80 components when following the paper's representation.
6. ML Models – DNN, SVR and KRR.
7. Evaluation – MAE, RMSE and R^2 .
8. Prediction – Voltage prediction for electrode materials.
9. Research Improvement – Additional feature, algorithm or application experiment.

Important Instructions to Students

⚠ IMPORTANT

Do not directly copy the paper's reported results into your project report. Implement the models, run the experiments and record your own results.

Your implementation should clearly distinguish:

- Original paper methodology — what the authors used.
- Your implementation — what you actually developed.
- Your improvement — what you changed or added.
- Your results — what you obtained from your experiments.

The paper concludes that ML can be used as a screening tool for electrode voltage prediction, while also noting the need for further improvements for routine application. Your project should report the limitations and validation of your own implementation accordingly.

► FIRST TARGET

Complete dataset collection, data inspection and feature generation before starting model training.